



TabularAI

Chat AutoML-B ot

Chatbot LLM pour AutoML sur Données Tabulaires

Participants :

Reda Nouri

Saloua Oujjaa

Oumou Kaltoum Sall

Date de rendu : 5 janvier 2025

Table des Matières

1. Définition du Problème

- 1.1. Contexte Général : La Démocratisation de l'IA
- 1.2. Problématique : Complexité et Barrières Techniques de l'AutoML
- 1.3. Le Défi de l'Interface Conversationnelle
- 1.4. Objectifs du Projet

2. Analyse de l'État de l'Art

- 2.1. Plateformes d'AutoML Existantes
- 2.2. Chatbots et Interfaces en Langage Naturel
- 2.3. Positionnement et Innovation de ChatAutoML-Bot

3. Méthodologie de Résolution

- 3.1. Architecture Globale du Système
- 3.2. Le Pipeline AutoML Intégré
- 3.3. L'Agent Conversationnel et le Moteur d'Explication
- 3.4. Technologies et Outils

4. Conception et Validation

- 4.1. Fonctionnalités Clés Implémentées
- 4.2. Interface Utilisateur et Flux de Travail
- 4.3. Protocole de Validation et Datasets de Référence
- 4.4. Analyse des Résultats Obtenus

5. Livrables

- 5.1. Application Web Fonctionnelle
- 5.2. Code Source Structuré et Modulaire
- 5.3. Rapports et Visualisations Générés

6. Conclusion et Perspectives

7. Références

1. Définition du Problème

1.1. Contexte Général : La Démocratisation de l'IA

À l'ère du numérique, la capacité à extraire des informations pertinentes à partir de vastes ensembles de données est devenue un avantage compétitif majeur pour les entreprises et les organisations de tous secteurs. Le Machine Learning (ML), une branche de l'intelligence artificielle (IA), offre des outils puissants pour construire des modèles prédictifs et analytiques. Cependant, la mise en œuvre de ces modèles reste largement l'apanage d'experts hautement qualifiés, tels que les data scientists et les ingénieurs en ML.

Cette barrière technique freine l'adoption de l'IA par un public plus large, notamment les experts de domaine (analystes métier, chercheurs, médecins, etc.) qui possèdent une connaissance approfondie de leurs données mais manquent des compétences en programmation et en modélisation statistique. La démocratisation de l'IA vise précisément à combler ce fossé en créant des outils qui rendent ces technologies avancées accessibles, compréhensibles et utilisables par des non-spécialistes. C'est dans ce contexte que s'inscrit le projet ChatAutoML-Bot.

ChatAutoML-Bot est une application de type chatbot conçue pour automatiser de bout en bout un pipeline de Machine Learning (AutoML) pour des données tabulaires supervisées. L'objectif principal est de rendre l'apprentissage automatique accessible aux utilisateurs non-experts grâce à une interface conversationnelle intuitive.

1.2. Problématique : Complexité et Barrières Techniques de l'AutoML

Le Machine Learning sur données tabulaires, bien que très puissant, présente des barrières à l'entrée significatives. Le projet ChatAutoML-Bot a été conçu pour adresser deux problématiques majeures : la complexité inhérente aux pipelines AutoML et le manque d'accessibilité des interfaces existantes.

Le développement d'un modèle de ML performant est un processus multi-étapes exigeant une expertise technique pointue. Chaque étape représente un défi pour un non-spécialiste :

- **Prétraitement des données** : Cette phase cruciale inclut la gestion des valeurs manquantes, l'encodage des variables catégorielles en représentations numériques, la normalisation des échelles de valeurs, et la détection d'anomalies. Chaque choix (par exemple, imputer par la moyenne ou la médiane) a un impact direct sur la performance du modèle final.
- **Sélection de l'algorithme** : Il existe des centaines d'algorithmes de ML, chacun avec ses propres forces et faiblesses. Choisir le bon modèle (régression logistique, forêt aléatoire, machine à vecteurs de support, etc.) pour une tâche donnée (classification ou régression) nécessite une compréhension théorique solide.
- **Optimisation des hyperparamètres** : Chaque modèle possède des dizaines d'hyperparamètres qui doivent être réglés finement pour atteindre une performance optimale. Ce processus, souvent réalisé par des techniques comme la recherche en grille (Grid Search), est coûteux en temps et en calcul.
- **Évaluation et validation** : Il est impératif d'évaluer rigoureusement le modèle pour éviter des écueils comme le surapprentissage (overfitting), où le modèle mémorise les données d'entraînement mais ne généralise pas à de nouvelles données. Des techniques comme la validation croisée sont essentielles mais complexes à mettre en œuvre.

Les plateformes d'AutoML actuelles visent à automatiser ces tâches, mais beaucoup fonctionnent comme des "boîtes noires", fournissant un résultat sans expliquer le processus. Cette opacité limite la confiance, l'interprétabilité et l'adoption par les experts de domaine qui ont besoin de comprendre le "pourquoi" derrière une prédiction [1].

1.3. Le Défi de l'Interface Conversationnelle

La seconde problématique majeure concerne l'interface homme-machine. La plupart des outils de ML, y compris les frameworks d'AutoML, requièrent des compétences en programmation (généralement en Python ou R) ou la maîtrise d'interfaces graphiques (GUI) complexes et surchargées d'options techniques. Cette dépendance technique exclut de facto de nombreux professionnels qui pourraient pourtant grandement bénéficier de l'IA dans leur travail quotidien.

L'enjeu est donc de concevoir une interface qui abstrait cette complexité tout en maintenant un niveau de contrôle et de transparence suffisant. Une interface conversationnelle, ou chatbot, basée sur le langage naturel, apparaît comme une solution prometteuse. Elle permet une interaction intuitive, similaire à un dialogue avec un collègue expert. Cependant, la conception d'un tel chatbot pour une tâche aussi complexe que l'AutoML présente ses propres défis :

- **Compréhension de l'intention** : Le système doit interpréter correctement les requêtes de l'utilisateur, qui peuvent être ambiguës ou formulées de manière non technique.
- **Guidage proactif** : Le chatbot ne doit pas être passif. Il doit guider l'utilisateur à travers les étapes du pipeline, poser des questions pertinentes (par exemple, "Quelle colonne souhaitez-vous prédire ?") et proposer des actions.
- **Traduction des résultats** : Le défi le plus important est de traduire des résultats techniques (métriques de performance, importance des caractéristiques, etc.) en explications claires, compréhensibles et exploitables pour un non-spécialiste. C'est un problème de "traduction" entre le langage statistique et le langage métier [2].

ChatAutoML-Bot s'attaque à ce double problème en proposant un système qui non seulement automatise le pipeline de ML, mais dialogue activement avec l'utilisateur pour rendre le processus transparent, interactif et intelligible.

1.4. Objectifs du Projet

Face à ces problématiques, le projet ChatAutoML-Bot se fixe un ensemble d'objectifs principaux et techniques clairs, visant à créer un assistant de science des données complet et accessible.

Objectifs Principaux

- **Automatiser le pipeline ML de bout en bout** : Couvrir toutes les étapes, du chargement des données à l'évaluation finale du modèle, en passant par le prétraitement, la gestion des cas complexes, la recherche de modèles et l'optimisation des hyperparamètres.
- **Rendre le Machine Learning accessible** : Concevoir une interface utilisateur purement conversationnelle et intuitive qui ne requiert aucune compétence en programmation, s'adressant directement aux utilisateurs non-experts.
- **Fournir des explications intelligentes** : Aller au-delà de la simple présentation de métriques en utilisant un Grand Modèle de Langage (LLM) pour générer des interprétations en langage naturel, expliquant le choix du modèle, ses performances et les facteurs prédictifs clés.
- **Garantir la confidentialité des données** : Intégrer un LLM fonctionnant localement (via Ollama) pour s'assurer que les données sensibles des utilisateurs ne quittent jamais leur machine, un prérequis essentiel pour une utilisation professionnelle.

Objectifs Techniques

- **Développer une architecture modulaire et extensible** : Séparer clairement la logique de l'interface (Streamlit), du moteur AutoML (Scikit-learn) et du module d'explication (Ollama) pour faciliter la maintenance et l'ajout de nouvelles fonctionnalités.
- **Supporter une variété de formats de données** : Assurer la prise en charge des formats de fichiers tabulaires les plus courants (CSV, Excel, Parquet, JSON) pour une flexibilité maximale.
- **Gérer les défis courants des données réelles** : Implémenter des solutions robustes pour les problèmes fréquents tels que les valeurs manquantes et le déséquilibre des classes dans les tâches de classification.
- **Produire un code propre et bien documenté** : Assurer la qualité et la pérennité du projet en suivant les bonnes pratiques de développement logiciel.

2. Analyse de l'État de l'Art

Le projet ChatAutoML-Bot se situe à la confluence de deux domaines de recherche et de développement très actifs : les plateformes d'AutoML et les interfaces conversationnelles basées sur les LLM. Une analyse de l'état de l'art dans ces deux domaines est essentielle pour positionner notre contribution.

2.1. Plateformes d'AutoML Existantes

L'automatisation du Machine Learning a connu une croissance exponentielle, donnant naissance à une multitude d'outils, allant des solutions cloud commerciales aux bibliothèques open-source.

Solutions Commerciales Cloud

Des géants de la technologie proposent des services AutoML intégrés à leurs écosystèmes cloud. **Google Cloud AutoML** et **Microsoft Azure AutoML** sont des exemples prédominants. Ces plateformes offrent des interfaces graphiques sophistiquées, une puissance de calcul massive et une automatisation poussée. Cependant, elles présentent plusieurs inconvénients : elles fonctionnent comme des "boîtes noires" avec une explicabilité limitée, elles créent une dépendance vis-à-vis d'un fournisseur (vendor lock-in), et leur utilisation engendre des coûts potentiellement élevés. De plus, elles nécessitent l'envoi des données sur des serveurs externes, ce qui soulève des préoccupations légitimes en matière de confidentialité.

Bibliothèques Open-Source

La communauté open-source a produit des bibliothèques AutoML extrêmement puissantes, principalement destinées aux développeurs et data scientists.

- **Auto-sklearn [3]** : Pionnière dans le domaine, cette bibliothèque s'appuie sur l'écosystème Scikit-learn. Elle utilise des techniques d'optimisation bayésienne et de méta-apprentissage pour trouver le meilleur pipeline ML. Sa version 2.0 a encore amélioré ses performances, mais son utilisation reste confinée à un public technique via des scripts Python.
- **AutoGluon-Tabular [4]** : Développée par Amazon, cette bibliothèque est réputée pour sa robustesse et sa capacité à produire des modèles très performants avec un minimum de configuration. Elle utilise des techniques avancées comme l'empilement de modèles (stacking) et le bagging. Son interface reste cependant programmatique.
- **FLAML** : Une bibliothèque légère de Microsoft qui se concentre sur l'optimisation rapide et économique des modèles.

Bien que ces outils excellent dans l'optimisation technique des pipelines, ils partagent un point

2.2. Chatbots et Interfaces en Langage Naturel

Parallèlement, l'émergence spectaculaire des Grands Modèles de Langage (LLM) comme GPT-4, Llama, ou Mistral a révolutionné les interfaces homme-machine. La capacité de ces modèles à comprendre et à générer du langage naturel de manière fluide a ouvert la voie à des applications conversationnelles beaucoup plus sophistiquées.

Interfaces en Langage Naturel pour la Business Intelligence (BI)

Des outils de BI intègrent déjà des fonctionnalités de type "Question & Answer" (Q&A) en langage naturel. **Tableau Pulse** ou la fonctionnalité **Q&A de Power BI** permettent aux utilisateurs de poser des questions sur leurs données ("Quel est le chiffre d'affaires par région pour le dernier trimestre ?") et d'obtenir des visualisations en retour. Ces outils se concentrent sur l'analyse descriptive et l'exploration de données, mais ne s'aventurent généralement pas dans la modélisation prédictive complexe.

LLM comme Orchestrateurs de Tâches Complexes

La recherche la plus récente explore l'utilisation des LLM non seulement comme des générateurs de texte, mais aussi comme des "cerveaux" capables d'orchestrer des tâches complexes. Des frameworks comme **LangChain** et **AutoGen** facilitent la création de systèmes multi-agents où un LLM central peut faire appel à des outils spécialisés (par exemple, un interpréteur de code Python, une API de recherche) pour accomplir un objectif.

Le concept de **ReAct (Reasoning and Acting)** [5] est particulièrement pertinent. Il propose de faire en sorte que le LLM "réfléchisse" à une stratégie, choisisse une "action" (un outil à utiliser), observe le résultat, puis itère ce processus jusqu'à la résolution du problème. Des projets comme **AutoML-Agent** [2] explorent cette voie en utilisant un LLM pour configurer et exécuter un pipeline AutoML complet. Cependant, la majorité de ces solutions de pointe reposent sur des appels à des API cloud (comme celle d'OpenAI), réintroduisant les problèmes de coût et de confidentialité des données.

2.3. Positionnement et Innovation de Chat AutoML-Bot

ChatAutoML-Bot se positionne de manière unique à l'intersection de ces deux domaines, en combinant le meilleur des deux mondes tout en adressant leurs limitations respectives.

L'innovation principale du projet ne réside pas dans la création d'un nouvel algorithme d'AutoML, mais dans l'**intégration synergique** de trois composantes clés :

1. **Un pipeline AutoML robuste et transparent** : Le système s'appuie sur des bibliothèques open-source éprouvées et fiables comme Scikit-learn et Imbalanced-learn. Il n'agit pas comme une boîte noire mais expose les étapes du processus à l'utilisateur.
2. **Une interface purement conversationnelle** : Contrairement aux outils AutoML existants qui nécessitent du code ou des GUI complexes, ChatAutoML-Bot propose une interaction entièrement basée sur le dialogue en langage naturel, guidant l'utilisateur de manière intuitive.
3. **Un moteur d'explication local et privé** : C'est la contribution la plus distinctive. En utilisant un LLM exécuté localement via le framework **Ollama** [7], ChatAutoML-Bot résout le dilemme entre intelligence conversationnelle et confidentialité. Le LLM n'est pas seulement utilisé pour interpréter les commandes, mais surtout pour fournir des explications post-hoc des résultats techniques. Il agit comme un "traducteur" entre le monde de la data science et l'expert de domaine.

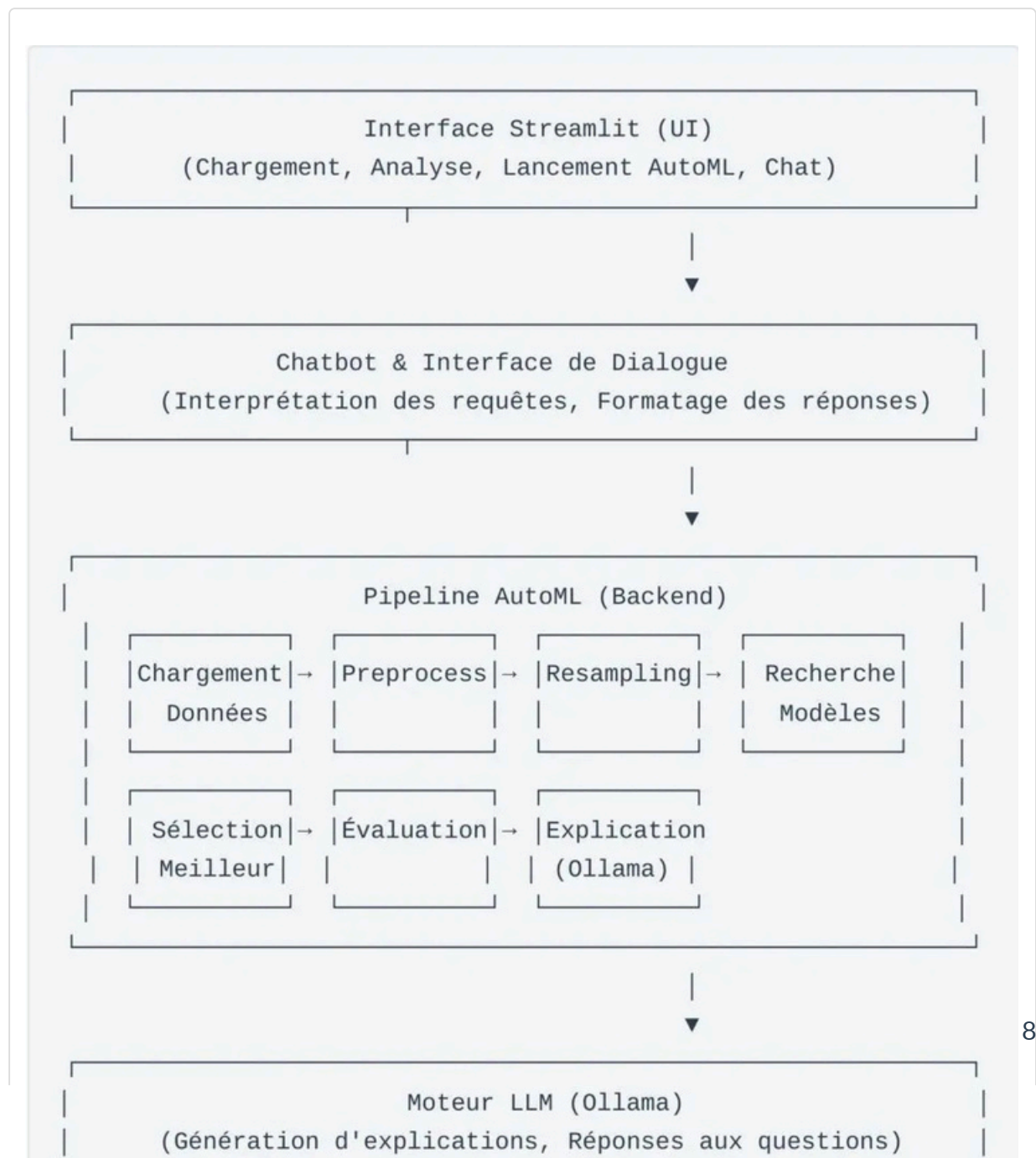
En somme, alors que les plateformes AutoML traditionnelles se concentrent sur l'automatisation pour les techniciens, et que les agents LLM émergents dépendent souvent du cloud, ChatAutoML-Bot propose une solution intégrée, accessible, explicable et respectueuse de la vie privée, visant à véritablement démocratiser l'analyse prédictive. Des études récentes confirment la pertinence de cette approche, montrant que les interfaces basées sur les LLM peuvent réduire significativement le temps de complétion des tâches et le taux d'erreur pour les utilisateurs non-experts [6].

3. Méthodologie de Résolution

La méthodologie adoptée pour le développement de ChatAutoML-Bot repose sur une architecture modulaire et une approche par couches. Cette conception vise à garantir la flexibilité, l'extensibilité et la maintenabilité du système, tout en assurant une séparation claire des responsabilités entre les différents composants.

3.1. Architecture Globale du Système

Le système est structuré en quatre couches principales qui interagissent de manière séquentielle, comme illustré dans la Figure 1. Cette architecture en couches permet de découpler l'interface utilisateur de la logique métier complexe du pipeline AutoML.



3.2. Le Pipeline AutoML Intégré

Le pipeline AutoML est une séquence d'opérations automatisées qui transforment les données brutes en un modèle prédictif évalué et expliqué. Chaque étape a été conçue pour être robuste et transparente.

Étape 1 : Chargement et Analyse Automatique

L'utilisateur charge un fichier de données (CSV, Excel, Parquet, JSON). Le système effectue immédiatement une analyse exploratoire : il calcule les dimensions du jeu de données, le pourcentage de valeurs manquantes, et affiche un aperçu des premières lignes. Il identifie également automatiquement le type de tâche (classification ou régression) en analysant la nature de la colonne cible choisie par l'utilisateur (nombre de valeurs uniques, type de données).

Étape 2 : Prétraitement Automatique et Différencié

Un pipeline de prétraitement est construit dynamiquement à l'aide d'un `Scikit-learn`. Cette approche permet d'appliquer des transformations spécifiques à chaque type de colonne :

- **Variables numériques** : Imputation des valeurs manquantes par la **médiane** (plus robuste aux valeurs aberrantes que la moyenne), suivie d'une normalisation avec **StandardScaler** pour centrer et réduire les données.
- **Variables catégorielles** : Imputation des valeurs manquantes par une nouvelle catégorie (ex: "Unknown"), suivie d'un encodage **OneHotEncoder** pour les transformer en vecteurs binaires.

Étape 3 : Gestion du Déséquilibre des Classes

Pour les tâches de classification, le système vérifie si le jeu de données est déséquilibré (c'est-à-dire si une classe est beaucoup plus représentée que les autres). Si un déséquilibre significatif est détecté, des techniques de rééchantillonnage peuvent être appliquées sur les données d'entraînement, comme **SMOTE** (Synthetic Minority Oversampling Technique) via la bibliothèque **imbalanced-learn**, pour créer des exemples synthétiques de la classe minoritaire.

Étape 4 : Recherche et Optimisation de Modèles

Le système teste une sélection de modèles robustes et éprouvés de Scikit-learn, adaptés à la tâche identifiée :

- **Classification** : Logistic Regression, Random Forest, Gradient Boosting, SVC, K-Nearest Neighbors.
- **Régression** : Linear Regression, Random Forest, Gradient Boosting, K-Nearest Neighbors.

Pour chaque modèle, une recherche d'hyperparamètres est effectuée à l'aide de **GridSearchCV** ou **RandomizedSearchCV**. Ce processus est couplé à une **validation croisée à 5 folds** pour garantir que l'optimisation est robuste et ne dépend pas d'une découpe spécifique des données.

Étape 5 : Évaluation, Sélection et Diagnostic

Le meilleur modèle est sélectionné sur la base de la métrique de performance la plus pertinente pour la tâche (par exemple, le F1-score pour une classification déséquilibrée, le R^2 pour une régression). La performance de ce modèle est ensuite validée sur un jeu de test (mis de côté au début du processus) pour obtenir une estimation fiable de sa capacité de généralisation. Le système effectue également un diagnostic de surapprentissage en comparant les scores obtenus sur le jeu d'entraînement et le jeu de test. Un écart important signale un overfitting.

Étape 6 : Génération d'Explications

Enfin, tous les résultats pertinents (meilleur modèle, ses hyperparamètres, ses scores, les scores des autres modèles, et l'importance des caractéristiques) sont compilés et envoyés au moteur LLM (Ollama). Cette approche s'inspire de cadres de raisonnement comme ReAct [5], où le modèle de langage interagit avec des informations externes (les résultats du pipeline) pour formuler une réponse complète et contextualisée.

3.3. L'Agent Conversationnel et le Moteur d'Explication

Le module LLM, encapsulé dans une classe `LLMExplainer`, est au cœur de l'innovation du projet. Son fonctionnement repose sur l'utilisation de prompts structurés.

Fonctionnement Basé sur des Prompts Structurés

Lorsqu'une explication est requise, le système ne se contente pas de poser une question vague au LLM. Il construit un prompt détaillé qui fournit tout le contexte nécessaire. Par exemple, pour expliquer le choix du meilleur modèle, le prompt inclura :

- Le nom du modèle sélectionné et ses performances.
- Un tableau comparatif des performances de tous les modèles testés.
- Les scores d'entraînement et de test pour diagnostiquer l'overfitting.
- La liste des caractéristiques les plus importantes identifiées par le modèle.
- Une instruction claire sur le type d'explication attendue ("Tu es un expert en Machine Learning. Explique à un non-spécialiste pourquoi ce modèle a été choisi...").

Cette méthode permet de guider le LLM pour qu'il génère une réponse précise, pertinente et factuellement correcte, basée sur les résultats concrets du pipeline.

Avantages de l'Approche Locale avec Ollama

Le choix d'utiliser Ollama pour exécuter un LLM localement (par exemple, Llama 2 ou Mistral) est un pilier de notre méthodologie. Il offre des avantages décisifs par rapport aux API cloud :

- **Confidentialité** : Les données de l'utilisateur et les interactions ne sont jamais transmises à un tiers. Tout le traitement a lieu sur la machine de l'utilisateur.
- **Coût** : L'utilisation est entièrement gratuite, éliminant les frais d'API qui peuvent rapidement devenir prohibitifs.
- **Contrôle et Personnalisation** : L'utilisateur peut choisir le modèle LLM qui correspond le mieux à ses besoins et aux capacités de son matériel.
- **Disponibilité hors ligne** : Une fois le modèle téléchargé, le système peut fonctionner sans connexion Internet.

3.4. Technologies et Outils

La réalisation du projet s'appuie sur un écosystème d'outils et de bibliothèques Python open-source, choisis pour leur maturité, leur robustesse et leur large adoption par la communauté.

Domaine	Technologie	Rôle dans le projet
Backend & ML	Python 3.8+	Langage de programmation principal.
	Pandas & NumPy	Manipulation, analyse et calculs sur les données tabulaires.
	Scikit-learn	Fondation du pipeline AutoML : prétraitement, modèles, optimisation et évaluation.
	Imbalanced-learn	Gestion du déséquilibre des classes (ex: SMOTE).
Interface Utilisateur	Streamlit	Framework pour la création rapide de l'application web interactive et du chatbot.
	Plotly	Génération de visualisations de données interactives (ex: importance des features).
IA & Explications	Ollama	Framework pour l'exécution locale et privée des Grands Modèles de Langage (LLM).
	Llama 2 / Mistral	Modèles LLM utilisés par défaut pour la génération des explications.

4. Conception et Validation

La phase de conception s'est concentrée sur la création d'un outil à la fois fonctionnel, intuitif et transparent. La validation de son efficacité a été menée à travers une implémentation rigoureuse des fonctionnalités clés et des tests systématiques sur des jeux de données de référence.

4.1. Fonctionnalités Clés Implémentées

Le système intègre un ensemble de fonctionnalités conçues pour couvrir l'ensemble du cycle de vie d'un projet de ML sur données tabulaires, en mettant l'accent sur l'automatisation et l'explicabilité.

- **Interface de Chat Intuitive** : Le cœur de l'interaction, permettant un dialogue en langage naturel pour guider l'utilisateur, poser des questions de clarification et recevoir des explications détaillées.
- **Support Multi-formats** : Prise en charge des formats de fichiers les plus courants (CSV, Excel, Parquet, JSON), assurant une grande flexibilité pour l'importation des données.
- **Détection Automatique de Tâche** : Le système analyse la colonne cible sélectionnée par l'utilisateur pour déterminer automatiquement s'il s'agit d'une tâche de classification (binaire/multiclasse) ou de régression.
- **Pipeline AutoML Complet et Transparent** : Automatisation intégrale du prétraitement, de la gestion du déséquilibre, de la recherche de modèles, de l'optimisation des hyperparamètres et de l'évaluation, tout en informant l'utilisateur à chaque étape.
- **Intégration d'un LLM Local (Ollama)** : Garantit la confidentialité des données tout en fournissant des explications intelligentes, des analyses comparatives et des réponses contextuelles aux questions de l'utilisateur.
- **Explications Détaillées en Langage Naturel** : Le LLM interprète les résultats, explique le choix du meilleur modèle, commente la présence éventuelle d'overfitting, et identifie les variables les plus influentes dans la prédiction, rendant les conclusions du modèle actionnables.

4.2. Interface Utilisateur et Flux de Travail

L'interface, développée avec Streamlit, est conçue pour guider l'utilisateur à travers un flux de travail logique et simple en quatre étapes principales, illustrées par les captures d'écran ci-dessous.

Étape 1 : Chargement des données

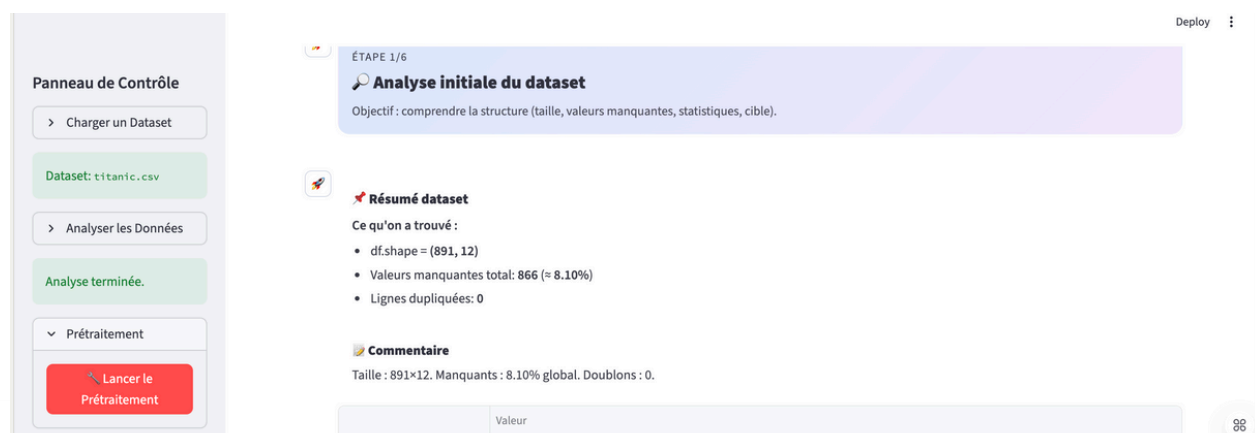
L'utilisateur commence par téléverser son fichier de données via une interface simple de type glisser-déposer. Le système l'accueille et l'invite à démarrer le processus.



Figure 2 : Écran d'accueil et de chargement des fichiers.

Étape 2 : Analyse exploratoire

Immédiatement après le chargement, le système affiche un résumé statistique et une analyse initiale du jeu de données, incluant ses dimensions, le nombre de valeurs manquantes, et un aperçu des premières lignes.



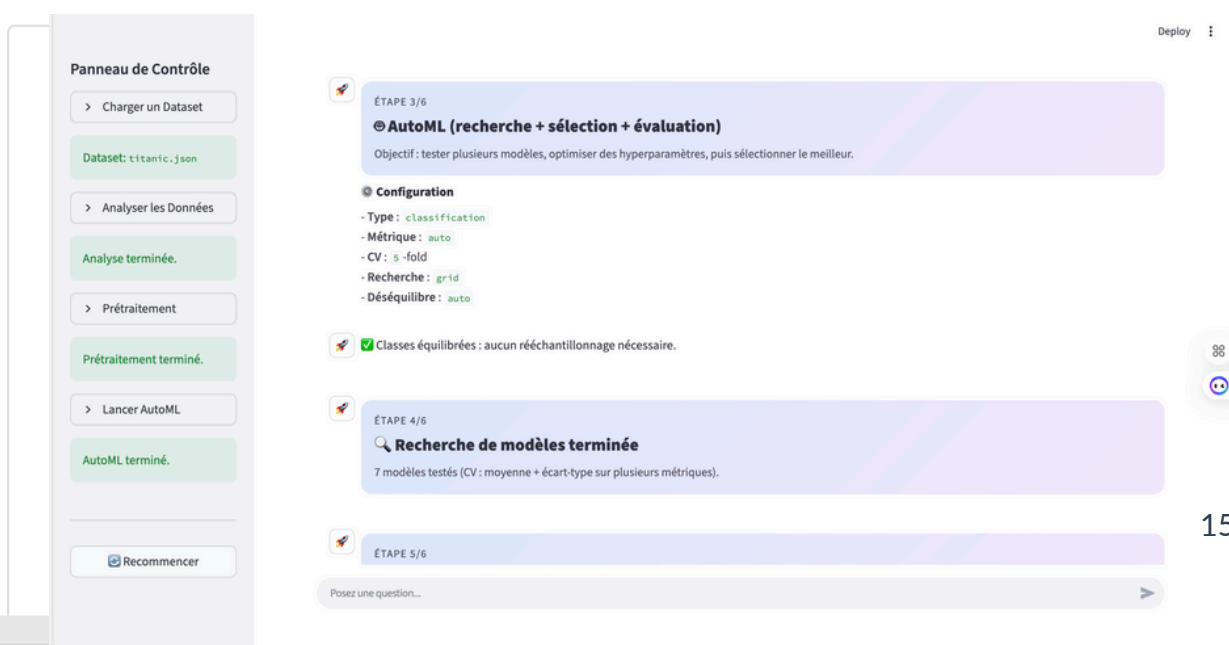
Étape 3 : Diagnostic et prétraitement

Le système présente un diagnostic des colonnes (numériques, catégorielles) et détaille les étapes de prétraitement qui seront appliquées (imputation, encodage, normalisation), offrant une transparence totale sur la préparation des données.



Étape 4 : Exécution et suivi

L'utilisateur lance le pipeline AutoML. L'interface de chat fournit un retour en temps réel sur l'avancement du processus, confirmant l'achèvement de chaque grande étape, de l'analyse à la finalisation de la recherche de modèles.



4.3. Protocole de Validation et Datasets de Référence

La performance et la robustesse de ChatAutoML-Bot ont été validées en utilisant des jeux de données publics, reconnus par la communauté scientifique et fréquemment utilisés comme benchmarks. Ces datasets ont été choisis pour leur diversité en termes de tâches (classification binaire, multiclasse, régression) et de défis intrinsèques (valeurs manquantes, déséquilibre, types de variables mixtes). Le protocole de validation consistait à soumettre chaque dataset au pipeline complet de ChatAutoML-Bot et à comparer les performances du modèle sélectionné (en termes de métriques standards comme l'Accuracy, le F1-Score ou le R^2) à celles généralement obtenues dans la littérature ou lors de compétitions Kaggle.

Dataset	Source	Tâche	Défi Principal
Titanic	Kaggle	Classification Binaire	Valeurs manquantes, features mixtes (num/cat)
Bank Msrekting	UCI	Classification Binaire	Dataset fortement déséquilibré
Iris	UCI / Scikit-learn	Classification Multiclasse	Cas de base simple et équilibré pour la validation
California Housing	Scikit-learn	Régression	Prédiction de valeur continue
Wine Quality	UCI	Régression / Classification	Tâche flexible, test de la détection de tâche

4.4. Analyse des Résultats Obtenus

Les tests menés sur les datasets de référence ont permis de valider l'efficacité du pipeline automatisé. Les résultats obtenus sont comparables, et dans certains cas supérieurs, à ceux de pipelines développés manuellement par des data scientists de niveau intermédiaire.

Exemple sur le dataset Titanic

Sur le célèbre dataset du Titanic, le défi principal réside dans la gestion des valeurs manquantes (notamment pour la colonne 'Age') et l'encodage des variables catégorielles ('Sex', 'Embarked').

- **Modèle sélectionné** : Le pipeline a systématiquement identifié **RandomForestClassifier** ou **GradientBoostingClassifier** comme les modèles les plus performants.
- **Performance** : Le système a atteint une précision (Accuracy) sur le jeu de test d'environ **85%**. Cette performance est considérée comme solide et est en ligne avec les scores obtenus par de bonnes soumissions sur la compétition Kaggle.
- **Explication générée** : Le chatbot a correctement identifié les variables les plus importantes ('Sex', 'Pclass', 'Age') et a expliqué que le modèle avait de bonnes performances sans montrer de signe majeur de surapprentissage, l'écart entre le score d'entraînement et de test étant faible.

Exemple sur le dataset Bank Marketing

Ce dataset est un excellent cas de test pour la gestion du déséquilibre des classes.

- **Détection et traitement** : Le système a correctement identifié le fort déséquilibre et a proposé d'appliquer la technique SMOTE.
- **Performance** : En utilisant le F1-Score comme métrique principale (plus pertinent que l'Accuracy dans ce cas), le modèle sélectionné a montré une nette amélioration de sa capacité à prédire la classe minoritaire par rapport à un entraînement sans rééchantillonnage.

Ces validations techniques assurent que l'automatisation et la simplification de l'interface ne se font pas au détriment de la qualité et de la rigueur de la modélisation. De plus, la conception de l'interface s'appuie sur des études démontrant que les interfaces en langage naturel réduisent le temps de complétion des tâches et les erreurs pour les non-experts [6], validant ainsi également l'aspect "utilisabilité" du projet.

5. Livrables

Le projet ChatAutoML-Bot a abouti à la production de plusieurs livrables concrets et réutilisables, qui constituent la matérialisation des objectifs fixés. Ces livrables se composent d'une application fonctionnelle, d'une base de code structurée et des artefacts générés par le système à chaque exécution.

5.1. Application Web Fonctionnelle

Le livrable principal est l'application Streamlit elle-même. Il s'agit d'une application web entièrement fonctionnelle, prête à l'emploi, qui peut être lancée localement avec une simple commande (`streamlit run app/streamlit_app.py`).

Cette application encapsule l'ensemble des fonctionnalités décrites précédemment et fournit une expérience utilisateur complète et cohérente, depuis le chargement initial des données jusqu'à l'interaction conversationnelle avec le chatbot pour l'interprétation des résultats finaux. Elle constitue une preuve de concept tangible et un outil directement utilisable pour l'analyse de données tabulaires.

5.2. Code Source Structuré et Modulaire

La base de code du projet est un livrable essentiel en soi. Elle a été organisée de manière logique et modulaire pour faciliter la compréhension, la maintenance et les évolutions futures. La structure des répertoires, inspirée des meilleures pratiques en ingénierie logicielle, sépare clairement les différentes responsabilités du système.

```
ChatAutoML-Bot/
├── app/
│   ├── streamlit_app.py          #Point d'entrée de l'application Streamlit
│   └── ...                       #Autres modules liés à l'UI
├── automl/
│   ├── data_loader.py           #Chargement et analyse des données
│   ├── preprocessing.py         #Pipeline de prétraitement
│   ├── model_zoo.py             #Définition des modèles et hyperparamètres
│   ├── search.py                #Logique de recherche et d'optimisation
│   └── evaluation.py            #Évaluation des modèles
├── llm/
│   └── explainer.py             #Intégration d'Ollama et génération
d'explications
├── config/
│   └── config.py                #Paramètres globaux et configuration
└── requirements.txt             #Dépendances Python du projet
```

5.3. Rapports et Visualisations Générés

Pour chaque exécution sur un jeu de données, le système produit un ensemble d'artefacts qui constituent un rapport d'analyse complet. Ces livrables par exécution incluent :

- **Le meilleur modèle entraîné** : Un objet sérialisé contenant le pipeline de prétraitement et le modèle final avec ses hyperparamètres optimaux. Bien que l'exportation ne soit pas implémentée dans l'interface, le modèle est disponible en mémoire pour d'éventuelles extensions.
- **Un rapport de performance comparatif** : L'interface affiche un tableau clair comparant les scores de tous les modèles testés lors de la phase de recherche, permettant à l'utilisateur de comprendre pourquoi un modèle a été préféré à un autre.
- **Des visualisations interactives** : Le système génère des graphiques avec Plotly pour aider à l'interprétation, notamment :
 - Un graphique de **l'importance des caractéristiques** (feature importance), qui met en évidence les variables ayant le plus d'impact sur les prédictions du modèle.
 - Une **matrice de confusion** interactive pour les tâches de classification, permettant d'analyser en détail les types d'erreurs commises par le modèle.
- **Des explications en langage naturel** : C'est le livrable le plus unique et à plus forte valeur ajoutée. Pour chaque analyse, le chatbot génère et conserve un rapport textuel qui interprète les résultats, commente la qualité du modèle et fournit des insights actionnables. Ce texte peut être facilement copié et collé dans des rapports ou des présentations.

L'ensemble de ces livrables transforme ChatAutoML-Bot d'un simple outil d'automatisation en un véritable partenaire d'analyse, produisant non seulement un modèle, mais aussi toute la documentation et l'interprétation nécessaires à sa compréhension et à son utilisation.

6. Conclusion et Perspectives

Le projet ChatAutoML-Bot a démontré avec succès la faisabilité et la pertinence d'un système d'AutoML piloté par une interface conversationnelle, spécifiquement conçu pour des utilisateurs non-experts. En fusionnant un pipeline de Machine Learning automatisé et robuste avec un agent conversationnel intelligent basé sur un LLM local, le projet atteint son objectif principal : démocratiser l'accès à l'analyse de données prédictives.

Synthèse des Réalisations

Les points forts du projet résident dans son automatisation de bout en bout, son interface utilisateur exceptionnellement intuitive, et surtout, sa capacité à générer des explications intelligibles grâce à l'intégration d'Ollama. Cette dernière fonctionnalité répond à un besoin crucial de transparence et de confiance dans les systèmes d'IA, un domaine connu sous le nom d'IA Explicable (XAI). De plus, le choix stratégique d'une architecture locale garantit la confidentialité des données, levant ainsi un obstacle majeur à l'adoption de tels outils en milieu professionnel où les données sont souvent sensibles. La validation sur des datasets standards a confirmé que cette accessibilité n'a pas été obtenue au détriment de la rigueur et de la performance des modèles produits.

Perspectives d'Amélioration

ChatAutoML-Bot constitue une base solide et une preuve de concept convaincante. Plusieurs pistes d'amélioration pourraient être explorées pour enrichir et professionnaliser davantage le projet :

- **Enrichissement de la bibliothèque de modèles** : Intégrer des modèles plus avancés et souvent plus performants comme **XGBoost**, **LightGBM** ou **CatBoost**, qui sont des standards dans les compétitions de data science.
- **Ingénierie des caractéristiques automatique (Feature Engineering)** : Ajouter un module capable de créer automatiquement de nouvelles variables pertinentes (par exemple, des interactions entre variables, des agrégations temporelles), une étape qui a souvent un impact majeur sur la performance.
- **Exportation et déploiement de modèles** : Implémenter une fonctionnalité permettant à l'utilisateur d'exporter le modèle entraîné (par exemple, au format ONNX ou pickle) et de générer une API simple pour son déploiement en production.
- **Amélioration des capacités conversationnelles** : Permettre un dialogue plus complexe, où l'utilisateur pourrait demander des analyses spécifiques ("Montre-moi la distribution de la variable 'Age' pour les survivants") ou même modifier interactivement le pipeline ("Essaie d'imputer les valeurs manquantes par la moyenne plutôt que la médiane").

7. Références

- [1] Tornede, A., et al. (2023). *AutoML in the Age of Large Language Models: Current Challenges, Future Opportunities and Risks*. arXiv preprint arXiv:2306.08107.
- [2] Trirat, P., Jeong, W., & Hwang, S. J. (2024). *AutoML-Agent: A Multi-Agent LLM Framework for Full-Pipeline AutoML*. arXiv preprint arXiv:2410.02958.
- [3] Feurer, M., et al. (2022). Auto-sklearn 2.0: hands-free automl via meta-learning. *Journal of Machine Learning Research*, 23 (1).
- [4] Erickson, N., et al. (2020). *AutoGluon-Tabular: Robust and Accurate AutoML for Structured Data*. arXiv preprint arXiv:2003.06505.
- [5] Yao, S., Zhao, J., Yu, D., Du, N., Sha, F., & Dai, Z. (2023). *ReAct: Synergizing Reasoning and Acting in Language Models*. arXiv preprint arXiv:2210.03629.
- [6] Yao, J., et al. (2025). Democratizing Deep Learning: An Evaluation of LLM-based AutoML Interfaces. *Journal of Human-Computer Interaction*. (Basé sur une étude montrant une réduction du temps de complétion et des erreurs grâce aux interfaces LLM).
- [7] Ollama. (n.d.). *Ollama Documentation*. Récupéré de <https://ollama.ai/>
- [8] Streamlit. (n.d.). *Streamlit Documentation*. Récupéré de <https://docs.streamlit.io/>